

Railway Depot Shunting

Assignment 24.1

Sets

- $t \in Types = \{1, 2\}$: unit types
- $u \in Units = \{1, 2, \dots, U\}$: units
- $k \in Tracks = \{1, 2, \dots, K\}$: tracks

Parameters

- $Arrival_u$: arrival time of unit u
- $Depart_u$: departure time of unit u
- $Length_u$: length unit u
- $TrackLength_k$: length of track k
- $Conflict[u_1, u_2]$ units u_1 and u_2 are in conflict
- $InDepot[u_1, u_2]$ if unit u_1 is in depot when unit u_2 arrives
- $Type_u$ type of unit u

Decision variables

- $x_{u,k} \in \{0, 1\}$: if unit u is assigned track k (=1) or not (0)

Model

Objective:

- Maximize the number of units that can be parked:

$$\sum_{u=1}^U \sum_{k=1}^K x_{uk}$$

Constraints:

- Each unit can be parked on at most track

$$\sum_{k=1}^K x_{uk} \leq 1 \quad \forall u = 1, 2, \dots, U$$

- Track length requirement

$$\sum_{j=1}^U InDepot[j, u] \cdot Length_j x_{jk} \leq TrackLength_k \quad \forall u = 1, \dots, U, \forall k = 1, 2, \dots, K$$

- Prevent track conflicts

$$x_{u_1 k} + x_{u_2 k} \leq \quad \forall u_1, u_2 | Conflict[u_1, u_2] = 1$$

The full model in Julia/JuMP is given below

```
using JuMP
using HiGHS
using Gurobi

include("unit_arrival_departure_info.jl")

Length = [42, 84]
trackLength = [500, 800, 700, 400, 650, 650, 750, 500]

K=length(trackLength)
U=length(Type)

RS=Model(Gurobi.Optimizer)

InDepot=zeros(U,U)

# identify whether two units are present in depot at same time
for u=1:U
    for j=1:U
        if arrival[j] <= arrival[u] && arrival[u] < depart[j]
            InDepot[u,j]=1
        end
    end
end
```

```

        end
    end
end

Conflict=zeros(U,U)

# calculate unit conflicts
for u=1:U
    for j=1:U
        if u < j
            if arrival[u]<=arrival[j]
                if depart[u] < depart[j] && depart[u] > arrival[j]
                    Conflict[u,j]=1
                end
            end
            if arrival[j]<=arrival[u]
                if depart[j] < depart[u] && depart[j] > arrival[u]
                    Conflict[u,j]=1
                end
            end
        end
    end
end

# unit track assignment
@variable(RS, x[u=1:U, k=1:K], Bin)

#maximize the number of units that can park
@objective(RS, Max, sum(x[u,k] for u=1:U for k=1:K))

# unit track assignment
@constraint(RS, [u=1:U], sum(x[u,k] for k=1:K)<=1)

# track length restriction (observe at each arrival)
@constraint(RS, [u=1:U, k=1:K], sum(InDepot[u,j]*Length[Type[j]]*x[j,k] for j=1:U)
    <= TrackLength[k])

# unit conflict constraints
@constraint(RS, [u=1:U-1, j=u+1:U, k=1:K; Conflict[u,j]==1], x[u,k]+x[j,k]<=1)

optimize!(RS)

if termination_status( RS) == MOI.OPTIMAL
    println("Number of Parked Units: $(objective_value(RS))")
end

```

```

for k=1:K
    print("Track $k:")
    unitsT = []
    for u=1:U
        if value(x[u,k])>1-1E-6
            push!(unitsT, (arrival[u],depart[u],Type[u]))
        end
    end
    sort!(unitsT, by = first)
    println(unitsT)
end
end
end

```

Assignment 24.2

New Variables

- y_{kt} track k is assigned units of type t

New Constraints

- Each track is assigned at most 1 type of unit

$$\sum_{t=1}^T y_{kt} \leq 1 \text{ for } k \in K$$

- Unit type track type consistency

$$x_{uk} \leq y_{kt} \quad \forall u \in U, k \in K, t \in T \mid \text{Type}[u] = t$$

The full model in Julia/JuMP is given below

```

using JuMP
using HiGHS
using Gurobi

include("unit_arrival_departure_info.jl")

Length = [42, 84]
trackLength = [500, 800, 700, 400, 650, 650, 750, 500]

K=length(trackLength)
U=length(Type)

```

```

T=length(Length)

RS=Model(Gurobi.Optimizer)

InDepot=zeros(U,U)

# identify whether two units are present in depot at same time
for u=1:U
    for j=1:U
        if arrival[j] <= arrival[u] && arrival[u] < depart[j]
            InDepot[u,j]=1
        end
    end
end

Conflict=zeros(U,U)

# calculate unit conflicts
for u=1:U
    for j=1:U
        if u < j
            if arrival[u]<=arrival[j]
                if depart[u] < depart[j] && depart[u] > arrival[j]
                    Conflict[u,j]=1
                end
            end
            if arrival[j]<=arrival[u]
                if depart[j] < depart[u] && depart[j] > arrival[u]
                    Conflict[u,j]=1
                end
            end
        end
    end
end

# unit track assignment
@variable(RS, x[u=1:U, k=1:K], Bin)

# track type choice (track is for units of type t)
@variable(RS, y[k=1:K, t=1:T], Bin)

#maximize the number of units that can park
@objective(RS, Max, sum(x[u,k] for u=1:U for k=1:K))

# unit track assignment

```

```

@constraint(RS, [u=1:U], sum(x[u,k] for k=1:K) <= 1)

# track length restriction (observe at each arrival)
@constraint(RS, [u=1:U, k=1:K], sum(InDepot[u,j]*Length[Type[j]]*x[j,k] for j=1:U)
    <= TrackLength[k])

# unit conflict constraints
@constraint(RS, [u=1:U-1, j=u+1:U, k=1:K; Conflict[u,j]==1], x[u,k]+x[j,k] <= 1)

# dedicated track constraints
@constraint(RS, [k=1:K, t=1:T], y[k,t] <= 1)
@constraint(RS, [u=1:U, k=1:K, t=1:T; Type[u]==t], x[u,k] <= y[k,t])

optimize!(RS)

if termination_status(RS) == MOI.OPTIMAL
    println("Number of Parked Units: $(objective_value(RS))")
    for k=1:K
        print("Track $k:")
        unitsT = []
        for u=1:U
            if value(x[u,k]) > 1-1E-6
                push!(unitsT, (arrival[u], depart[u], Type[u]))
            end
        end
        sort!(unitsT, by = first)
        println(unitsT)
    end
end
end

```